Case Study Report

COM579001P6 – Big Data Processing

2802410346 - JULIUS GILANG NUGROHO

Big Data Architecture for Real-Time Air Quality Monitoring in Brooklyn, New York

Contents

Cover Page 1

Executive Summary 2

A. Introduction 2

B. Big Data Architecture Design 4

C. Implementation Details 12

D. Result and Discussion 21

E. Conclusion 21

F. Declaration Statement 23

Reference 24

Executive Summary

Air quality monitoring is an important issue in urban areas such as Brooklyn, New York, because poor air quality can cause negative effect of public health. Since IoT based air quality sensors continuously generate large amounts of PM 2.5 data, traditional systems may cause struggle to process the efficiency data in real time. This project focuses on how Big Data Technologies can use to store, process, and analyse air quality data to make an accurate decision making.

The project uses the Air Quality Data in Brooklyn, NY (OpenAQ 2025) dataset from Kaggle, which contains PM 2.5 measurements, timestamps, geographic coordinates, and station information collected from multiple sensors. Because the data is generated continuously over time, it is categorized as time series Big Data.

The proposed architecture uses a layered Big Data approach deployed on a public cloud environment. Real time sensor data is ingested through a streaming pipeline, stored using Apache Hadoop HDFS and Apache HBase, and processed using Apache Spark and PySpark. The analysis shows that most PM 2.5 readings fall into the good and Moderate categories, with higher pollution level appearing during certain hours. A Random Forest Classification was used to predict air quality categories based on location data and time.

Overall, the project demonstrates how Big Data Technologies can support to scalable the storage and monitoring in real time for smarter public health and urban management.

A. Introduction

1. Background

- Dataset Source: https://www.kaggle.com/datasets/aidenl25/air-quality-data-in-brooklyn-ny-openaq-2025
    - This dataset is sourced from Kaggle with topic “Air Quality Data in Brooklyn, NY – OpenAQ 2025”. This data contains air quality records, specifically about air quality level on PM 2.5 parameter, timestamps, and locations from many monitoring stations in Brooklyn, New York area.
- Business Domain and Problem Statement
    - Urban areas air pollution has become a major public health concern, especially in densely populated city where pollution levels can change quickly especially by traffic, industrial activities, and weather conditions. The main challenge is that traditional data processing systems are often unable to handle the large amount of real time data generated continuously by IoT based air quality sensors. Then, a scalable Big Data system is need to make an efficiency collect, store,

and analyze data in real time. A system can help detect pollution increases, identify high risk areas with poor air quality, and support faster decision making to improve environment monitoring and public health awareness.

2. *Big Data Characteristics*

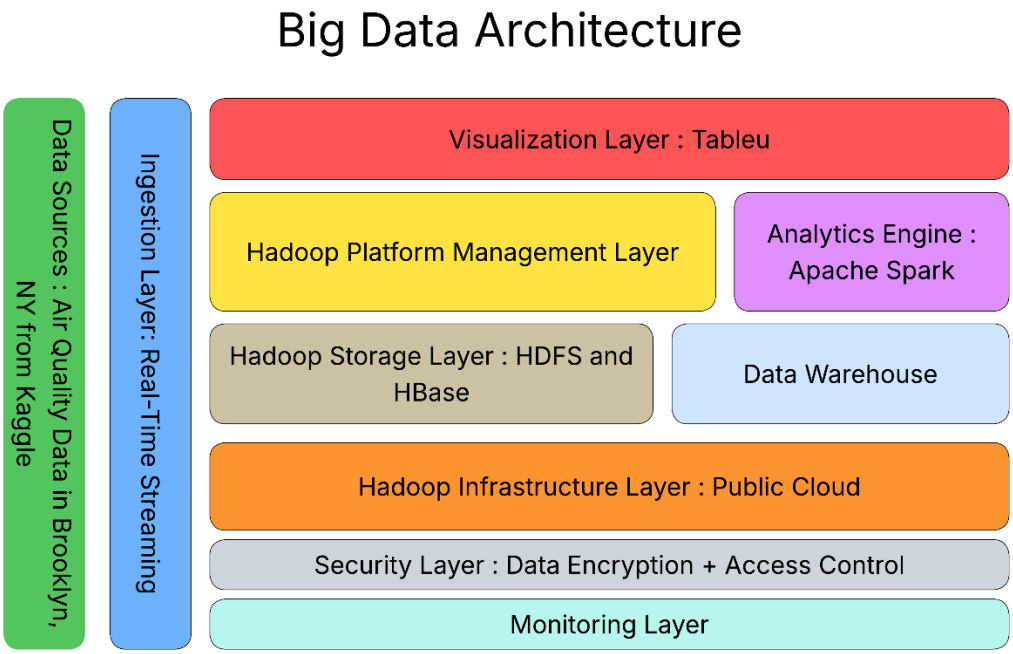
- Volume: The dataset contains around 1,391 hourly PM 2.5 records collected from monitoring stations in Brooklyn. Although relatively small, it represents a real world smart city scenario where IoT sensors continuously generate large amounts of environmental data require scalable Big Data Storage systems.
- Velocity: Air quality sensors generate new PM 2.5 in every hour, creating continuous real time. This high velocity data requires streaming and real tim processing infrastructure.
- Variety: The dataset includes different data types such as numerical values, timestamps, geographic coordinates, and categorical information like sensors providers. Real world system may also add external data like weather and traffic.
- Veracity: The dataset contains data quality issues that found in IoT systems, including missing values, inconsistent data types, and duplicate rows. This topic highlight about importance data cleaning and validation before analysis.
- Value: After processing and analysis, then data can make a useful insights for government, healthcare, and the public to support monitoring pollution, and improve public health awareness

3. *Business Motivation*

- Limitations of Traditional Data Processing
    - Traditional Database have limited scalability and are not suitable for handling many realtime IoT sensor stream efficiently. Traditional system have struggling to process semi structured data like JSON and geographic information, make Big Data Technologies are suitable for this case.
- Business Motivations and Expected Value
    - Implementing a Big Data Architecture allows government and organizations to make more accurate data driven decision during high pollution. The system supports pollution forecasting, better resource allocation and more sustainable urban area planning.

B. Big Data Architecture Design

1. Architecture Overview



2. Data Sources

The main data source used in this architecture consists of two CSV files

- **Measurements.csv** contains 1,391 hourly PM 2.5 readings collected from two monitoring stations between August 1- 30, 2025. Each record include sensor details, timestamps, coordinates, and PM 2.5 values in µg/m³. Data quality issues found PM 2.5 column being stored as a string, missing values, in isMobile, isMonitoring, and country_iso, and a duplicate header row caused by data merging.
- **Stations.csv** provides metadata for the two monitoring stations, including station ID, station name, coordinates, sensor type, and data source. Station 1 (Near Bay 50 St) uses in AirGradient sensor, while Station 2 (Bklyn – PS 314) uses a reference grade AirNow monitor.

In a real production environment, the system can expanded by integrating other data sources like real time OpenAQ API stream, weather APIs, NYC traffic data, and satellite aerosol measurement to improve air quality analysis.

3. *Ingestion Layer*

Ingestion layer in this project used streaming type because the sensors produce data continuously in real time. So the data will be collected at the time the data recieved.

Also the pattern that are used is realtime streaming pattern. It is really good because this problem used IoT monitoring system, and PM2.5 measurement are generated continuous and need to be processed immediately to produce decision solution as soon as possible. An effective ingestion component need to have ability to handling many data and fault tolerant data from many IoT sensors together. So no data will lost during continuous transmission [3].

By using a real-time streaming ingestion pattern, the system is able to:

- Process continuous sensor data efficiently,
- Detect sudden pollution spikes in real time,
- Support live monitoring dashboards,
- Generate real-time alerts when PM2.5 levels reach dangerous

4. *Storage Layer*

The storage layer in this architecture combines Hadoop Distributed File System (HDFS) and Apache HBase to manage big air quality sensor data in a scalable, efficient, and fault-tolerant environment.

HDFS (Hadoop Distributed File System)

HDFS role is becoming the primary distributed storage system for raw and modified data. Since the sensors continuosly generate data and the volume of data become more bigger over time, HDFS is really suitable, because HDFS can strongly deal with fault tolerance, more scalable, and cost efficiency [1]. So HDFS really good with large scale batch processing and modern big data environment.

So, HDFS is suitable for:

- Storing large historical datasets from long monitoring periods,
- Distributing storage across multiple nodes,
- Supporting batch analytics for long-term trend analysis

HBase

Apache HBase is a NoSQL database that was built on HDFS, help fast real time read and write operations for continuous IoT sensor data. HBase support real time access to sensor generated data, making it really suitable with this project [5]. This ability help system to quickly respond to environmental event while dealed with dashboard and give automated notifications.

Stanic *et al.* [11] explain that HBase uses a four-dimensional data model consisting of row keys, column families, column qualifiers, and data value versions. This structure is well suited for high-velocity and time-stamped sensor data generated by IoT monitoring systems.

HBase supports efficient handling of:

1. High-velocity PM2.5 sensor data ingestion,
2. Fast retrieval of current sensor readings,
3. Low-latency access to time-series data,
4. Scalable processing across monitoring stations.

Why HDFS + HBase?

Combination of HDFS and HBase creates a good storage architecture because:

1. HDFS provides scalable and fault-tolerant storage for massive historical datasets [1][7],
2. HBase help real-time access to live sensor data [5][11],
3. Both technologies support horizontal scalability for growing sensor deployments,
4. Together, they can efficiently process continuous IoT streaming data from the real-time ingestion pipeline [4].

5. *Processing & Analytics Engine*

The analytics engine uses Apache Spark for main processing framework for handling large scale air quality sensors data. Spark is chosen because it supports batch processing and realtime streaming analytics within a single platform, make it suitable for processing and continuous PM 2.5 sensor data.

Apache Spark as a Unified Analytics Engine

Apache Spark is open source distributed analytics engine for large scale data processing. Spark used because flexible and capable of handling both batch and stream processing with scalable performance for high volume Big Data workloads [12]. The advantages is in memory processing architecture, which reduces latency and provides faster computation compared to traditional like Hadoop MapReduce, while still supporting when memory capacity is exceeded [12]. Spark become a framework for Big Data Analytics because there are libraries for machine learning, streaming, graph, and structured data processing [8].

Spark Structured Streaming for Real-Time Processing

Realtime component of this air quality monitoring system, Apache Spark Structured Streaming is used for process continuous PM 2.5 sensor data. According to Anand et al. [4],

Spark Structured Streaming is scalable, high throughput, and fault tolerant stream processing framework that applies a micro batch processing model to process incoming data. This important for air quality monitoring because sensor data have to be processed faster to support real time update and fast pollution detection. Spark Structured Streaming collect data from many sources, including Apache Hadoop HDFS and Apache HBase, ensuring integration between storage and analytics layer.

Spark Integration with HDFS and HBase

Apache Spark have another advantage is compatibility with the storage layer. According to AWS [3], Spark not provide its own storage system, but perform analytics directly on external storage platform like Apache Hadoop HDFS by leveraging YARN to share the same cluster and dataset. Spark on Hadoop support the integration of compute and storage resources, enable an effective processing historical in HDFS and realtime in Apache HBase.

Why Apache Spark?

- Apache Spark supports batch and realtime streaming analytics that making it suitable for processing historical and continuous PM 2.5 sensor data [12].
- Spark uses in memory processing model that provides faster computation and lower latency compare traditional system like Hadoop MapReduce.
- Spark Structured Streaming enable scalable, low latency, and fault tolerant processing continuous air quality sensor for real time monitoring and pollution detection [4].
- Spark integrate with Apache Hadoop HDFS and Apache HBase, ensuring integration between storage and analytics [3].
- Spark supports horizontal scalability distributed nodes, and allowing system to handle future growth in number of IoT monitoring sensors [12][8].

Overall, Apache Spark provides scalable and reliable analytics engine for historical and real time environmental monitoring in smart city applications.

6. Visualization Layer

To visualize data in this architecture, tableau used as the primary tool to produce data insights to users. Tableau selected beacuse it provides an interactive and user friendly interface that help analysts and stakeholders to explore the trend, distribution graph, and pattern from the sensor data.

In this system, tableau will visualize processed data from spark, where the sensor raw data has been cleaned, aggregated, and structured. So the insight that displayed in tableau shows data that is ready for analysis to make the dashboard precise and insightful.

7. Monitoring & Security (Conceptual)

Security Layer

The security layer in this architecture designed to protect the air quality sensor data in the big data system (ingestion, storage, etc). Since the data flows through many distributed technologies like HDFS, HBase, and spark, securing stored data and transformed data is needed to prevent from unauthorized access, and many other threats.

- **Data Encryption**

To secure the PM2.5 sensor data, encryption is used in storage and transmission. Data stored in HDFS is protected by Hadoop Transparent Data Encryption. Also transfering data between node secured by SSL/TLS encryption. SSL/TLS help encrypt communication between cluster nodes and clients, help reduce the risk of threats during transmission [10].

- **Access Control**

Access control based on role is used in this system to manage permission of other contributor. This make sure only authorized contributor can access specific/important data and system functions. For example, the analytics engine allowed to process data from HDFS, while external users can only access the result through dashbords.

Overall, security layer helps to make sure the air quality data safe and protected in the big data pipeline end to end.

Monitoring Layer

Monitoring layer in this system has functions to continuosly monitoring the performance and reliability of all components involved in processing the air quality data, including the storage tool, analytics engine, and overall data pipeline. Since the data is processed realtime continuously, there must be failure or issues. So this layer help locate the issue to avoid data loss, error, or delayed analysis.

Cluster and Infrastructure Monitoring

System monitors the health of the hadoop cluster, including CPU usage, memory usage, disk capacity, and many others activity in nodes. Effective Hadoop monitoring need real time observation tools that provide visibility to clusters, jobs, and nodes. Many tools that help this process like Apache Ambari and Prometheus can help collect metrics from HDFS and diplay it on monitoring dashboards [2].

Performance Alerting

Realtime alerts also used to alert the admin when there are critical performance issues such as high memory usage, low HDFS storage, failed spark jobs. System health, system integrity and CPU [9] should be monitored by a monitoring system at all times. Alert system such as prometheus can help to send notifications in real-time.

Centralized Log Collection

Generated logs from different components and HDFS operations, HBase, and spark job, and collect it into centralized log system. Tools like ELK stack can be used to collecting, indexing, and analyzing logs from many cluster, making detcting issue and troubleshooting more efficient.

Overall, monitoring systems have functions to keep the infrastructure reliable, observable, and responsive by real time monitoring, allerting, and logging.

8. Infrastructure Decision Analysis

- **Streaming Ingestion.** Since PM2.5 sensor data is generated continuously and needs to be processed in realtime, streaming ingestion is the most suitable approach for this air quality monitoring system.
- **HDFS + HBase.** Apache Hadoop HDFS is employed to store large quantities of data for batch processing at a later date while Apache HBase is used to retrieve data for realtime queries with a high speed. The combination of both technologies enable the system to achieve both long term archival and low latency realtime data access simultaneously.
- **Apache Spark.** The reason for choosing Apache Spark over Hadoop MapReduce is because of the ability to do faster in-memory processing and better support for realtime analytics via Structured Streaming. This will make Spark more suited for processing historical sensor data and continuous streams of PM2.5 data.

- **Tableau.** The use of Tableau is chosen as it has the ability to directly connect to processed analytics outputs and build interactive visualisations without extensive development. This makes the insights in air quality clearer to understand for the non-technical stakeholders.

Deployment Model

This deployment model for this big data architecture is public cloud, where all infrastructure elements, such as storage, analytics, visualization are deployed and managed by a third party cloud provider Google Cloud Platform.

Why Public Cloud?

For this project, the most convenient approach will be a public cloud deployment model, since it does not involve the management of physical hardware infrastructure. The system is designed on the basis of the Air Quality Data in Brooklyn, NY (OpenAQ, 2025) data set from kaggle, so no servers or data centers are needed to purchase or maintain. Cloud computing can be used to provision all computing resources on demand, enabling the project team to concentrate on developing and managing the data pipeline, without having to worry about keeping the infrastructure up to date. Major cloud providers also provide some managed services across the Hadoop ecosystem like Amazon EMR and Google Cloud Dataproc, which make deployment and processing with Apache Spark and system management easier.

The other big benefit of moving to a public cloud is scalability and cost-saving. Air quality sensor data is generated and is added over time, particularly in time-series data where historical data is added over long periods of time. The public cloud platforms allow for dynamic storage and computing resources to be provisioned to meet workload requirements, which is very appropriate for large-scale analytics processing. At the same time, the pay-as-you-go pricing model ensures that costs are incurred only for the resources being used. This is more cost-efficient than developing and maintaining infrastructure on-site, for academic and research projects.

Plus, public cloud deployment also enhances accessibility and collaboration between stakeholders. The system and analytical dashboards can be accessed via the internet, allowing researchers, city analysts and environmental agencies to monitor and analyze air quality data in various locations. This flexibility enables greater cooperation and faster information transmission, as well as enhanced decision-making. Air quality trends and analytical results are presented in a user-friendly and interactive format, with the ability to integrate visualization platforms like Tableau.

To sum up, the Public Cloud deployment model is the most practical and effective for this architecture as it does not depend on any hardware, is able to scale as the volume of sensor

data continues to grow, lowers operational costs and ensures wide accessibility which is a perfect match with the goals of this air quality monitoring project.

Virtual Infrastructure vs. Physical Data Center - Implications for This Architecture

One of the primary design choices in this project is whether the big data infrastructure should be implemented on virtual infrastructure (on cloud) or physical data center (on premise). This architecture is designed for a public cloud deployment model, so the performance and scalability of Hadoop Distributed File System, Apache HBase, and Apache Spark in the real world have significant implications.

A physical data center requires the organization to purchase, install and maintain all hardware resources such as servers, storage gear and network gear. While this gives the full control over the infrastructure, it also slows down the scalability as the capacity is increased, the number of physical machines and infrastructure configuration increases, making it more costly. This can be a significant restriction for a project that is constantly ingesting ever increasing air quality time-series data from Brooklyn, especially if a large historical dataset needs reprocessing, or if several Spark jobs need to be executed at once. To meet the growing storage and computational requirements of the organization, they would have to make significant investments in time and resources.

Unlike a cloud-based virtual infrastructure, which abstracts the underlying physical hardware, a virtual infrastructure enables computing resources, like CPU resources, memory, and storage, to be dynamically provisioned within minutes. This scaling flexibility allows for adding virtual nodes to HDFS clusters as storage demands grow, and scaling them back when the workloads are not as analytics-intensive, helping to balance cost.This scaling flexibility also allows for scaling HDFS clusters up to the number of virtual nodes required to meet storage needs, and scaling them back when the workloads are not as analytics-driven, helping to balance cost. Furthermore, cloud platforms offer inherent redundancy and fault tolerance within each availability zone, ensuring higher reliability of the overall system. Together with HDFS data replication, the system is more resilient without the need to manually manage complex physical infrastructure.

The downside of virtual infrastructure is that it is less dependent on hardware that can be managed and less flexible with regard to pricing and network performance policies set by the cloud provider. But when it comes to conducting research and monitoring, virtual infrastructure offers more benefits

than a traditional physical data center in terms of flexibility, scalability, deployment speed and cost effectiveness.

Hadoop's Infrastructure Assumptions Relevance to This Project

There were some infrastructure assumptions that were the original basis for Apache Hadoop. Hadoop will run on clusters of commodity hardware, which are relatively low cost, and where hardware failures can be assumed to happen. Consequently, the overall Hadoop ecosystem, such as Hadoop Distributed File System replication, the fault tolerant scheduling in YARN, and distributed processing frameworks like Apache Spark, assume that node failures are not the exception.

As the air quality monitoring system is deployed in a public cloud platform rather than commodity servers, these assumptions are still valid for this project.

Replication of data for reliability is one of the important assumptions. The data should be replicated in HDFS across a number of nodes, typically three replications are used, to guarantee data availability if a node fails. In this project, PM2.5 sensor data is historical time-series data that cannot be recovered. Thus, if any data were to be lost, it would be lost forever. HDFS' replication feature helps ensure integrity and availability of the Brooklyn air quality data in the distributed cloud.

Horizontal scalability is another important assumption. Hadoop focuses on the principle that it is better to add more nodes to the system than upgrade one node. Given the fact that the amount of air quality sensor data is continually growing over time, this horizontal scaling approach is very well suited for this project. The architecture can scale by adding a virtual cloud node for each additional monitoring station or by adding more historic data to existing nodes, without making significant changes to the architecture.

Overall, the key principles of the infrastructure of Hadoop, such as using commodity-based distributed systems, employing fault tolerance with replication, and scaling horizontally across multiple clusters continue to be extremely relevant to this project. Cloud-based virtual infrastructure adds to these principles. Simultaneously, Hadoop's inherent inability to process data in realtime has been overcome with the addition of Apache HBase and Apache Spark, resulting in a manner of doing things that retains the key concepts of Hadoop and satisfies the needs of current environmental monitoring needs for IoT.

C. Implementation Details

Big Data Analytics

The main analytics of this project used Apache Spark on google colab. With this tools, there are two data that were processed, measurement.csv file that contain PM2.5 sensor details, and stations.csv that contain stations data. The analytics process do data preprocessing, descriptive analytics, and classification modeling.

Descriptive Analytics

There are three descriptive analytics in this project that are being researched to understand the condition of PM2.5 levels.

Pollution Level Distribution. This dataset main analytics sources is pollution_level that are used to categorize every condition into three category such as Good, Moderate, and Unhealthy. So it help to alert and give insigth frequency when the air quality reach bad category. The visualization used pie chart to show informartion more clear and easy to understand.

Average PM2.5 by Provider. Data from sensors are grouped by data provider and then averaged to determine if there are any trends among the data providers that correspond to either different sensor stations or different data sources that consistently report higher or lower PM2.5 concentrations. The result is displayed in a bar chart, ordered from highest to lowest to show the highest polluters.

Daily Average PM2.5. All sensor data is used to determine daily average PM2.5 levels to investigate the variation in air pollution levels during the month. A bar chart is used to display the results for days 1 to 30. The analysis shows that PM2.5 levels are highest on day 5 and day 6, at about 35 µg/m³ and 26 µg/m³, both of which are considered to be in the moderate to unhealthy pollution category. Pollution levels then slowly drop in the second half of the month. This analysis is used to determine the days that have the most severe air pollution in Brooklyn during the observation period.

Classification Model — Random Forest Classifier

PySpark is used to train Random Forest Classifier to predict the pollution_level category (Good, Moderate or Unhealthy) for each sensor reading, using temporal and geographic features.

AQI Category	AQI Value	Average $PM_{2.5}$ Concentration (µg/m³)
Good	0 - 50	0 - 15.4
Moderate	51 - 100	15.5 - 40.4
USG	101 - 150	40.5 - 65.4
Unhealthy	151 - 200	65.5 - 150.4
Very Unhealthy	201 - 300	150.5 - 250.4
Hazardous	301 - 500	250.5 - 500.4

We classified the pollution level into three categories based on this source. Values above the moderate threshold were labelled as unhealthy.

Features used:

- Day: Day of the measurement was taken.
- Month: Month of the measurement
- Hour: Hour of the measurement

- Latitude: Geographic latitude of the sensor station.
- Longitude: Geographic longitude of the sensor station.

The pollution_level label is first converted to numeric value using StringIndexer. All features are then combined into a single vector with the VectorAssembler. To ensure the reproducibility of the results, the data set is split into 80% training and 20% testing sets with a fixed random seed of 42. Then the Random Forest model is trained with the training set and predictions are made on the test set.

There is a major issue in the dataset. The classes are highly imbalanced across categories. The “unhealthy” class has significantly fewer samples compared to the others. To address this, SMOTE applied to balance the dataset, resulting in class distributions of 852 for Good, 600 for Moderate, and 300 for Unhealthy.

```
paramGrid = (
    ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100, 150])
    .addGrid(rf.maxDepth, [5, 10, 15])
    .build()
)
```

```
print("Best NumTrees:", bestModel.getNumTrees)
print("Best MaxDepth:", bestModel.getMaxDepth())
```

```
Best NumTrees: 50
Best MaxDepth: 15
```

```
cv = CrossValidator(
    estimator=rf,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=5,
    seed=42
)
```

Then hyperparameter tuning performed on the RandomForestClassifier to find the optimal configuration. The parameters tested were numTrees (50, 100, 150) and maxDepth (5, 10, 15). After training and evaluation, the best-performing model used 50 numTrees and a maxDepth of 15.

Also we use 5-Fold Cross Validation to identify the optimal Random Forest parameters.

Overall, Hyperparameter tuning with 5-fold cross validation selected a model that generalized better across multiple validation folds. Although the final test accuracy was slightly lower, the difference was minimal and the model selection process became more robust and less dependent on a single train-test split.

Evaluation Metrics

The trained model is evaluated using two metrics from MulticlassClassificationEvaluator such as Accuracy and F1 Score

```
for feature, importance in zip(
    feature_cols,
    bestModel.featureImportances
):
    print(feature, ":", importance * 100, "%")
```

```
latitude : 9.202943342971317 %
longitude : 9.524309677326432 %
month : 0.0 %
day : 64.8936760855728 2 %
hour : 16.3790708941294 3 %
```

```
print("Accuracy:", accuracy)
print("F1 Score:", f1)
```

```
Accuracy: 0.9541666666666667
F1 Score: 0.9551328864165833
```

label	prediction	count
0.0	0.0	167
0.0	1.0	5
1.0	0.0	3
1.0	1.0	56
1.0	2.0	3
2.0	2.0	6

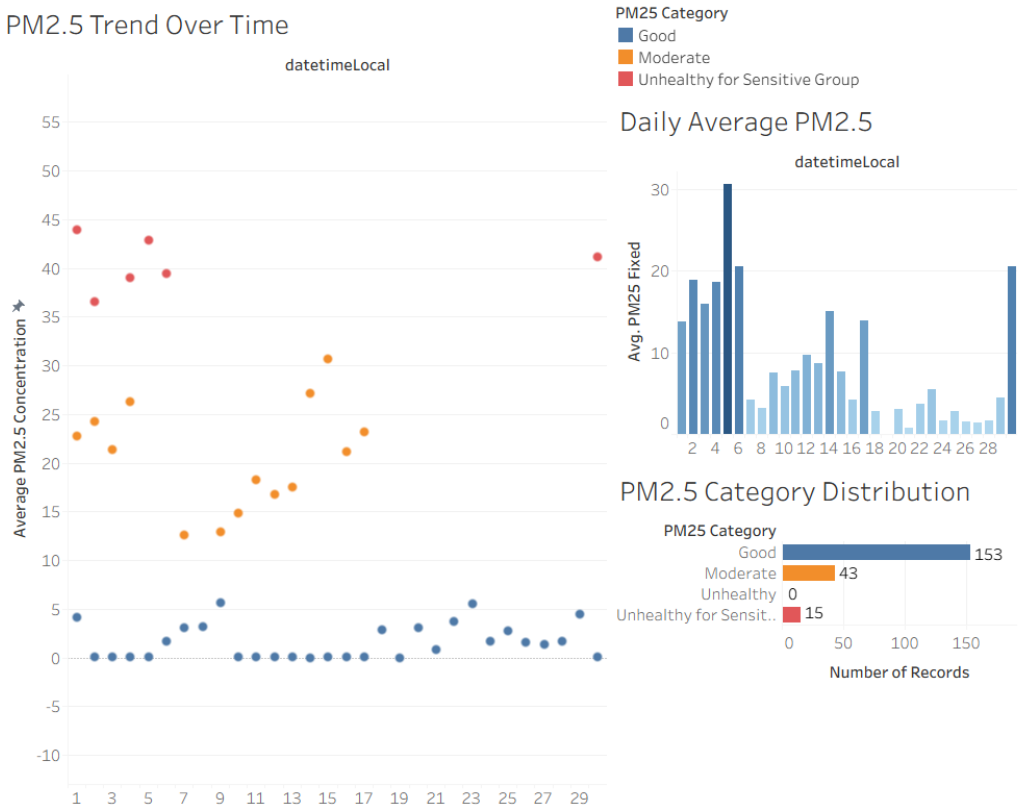
Interpretation of Results

The descriptive analytics shows that the most PM2.5 readings in the Brooklyn dataset fall within the Good to Moderate range, suggesting that air quality in the area being monitored is generally good for most of the monitoring period. However, the distribution chart shows that there are Unhealthy readings, which indicates that spiking pollution should be monitored.

The peak pollution hour analysis can help identify specific hours of the day during which PM2.5 concentrations are regularly high and can be used to develop time-sensitive environmental policy measures, such as traffic restrictions or limits on industrial operations, during high-pollution hours.

The Random Forest Classifier shows that the hour of the day, month, and location of the sensor do have meaningful signals in predicting the air quality category. The three pollution categories are classified by using multiple decision trees in the ensemble, which decreases overfitting and yields a more robust classification surface.

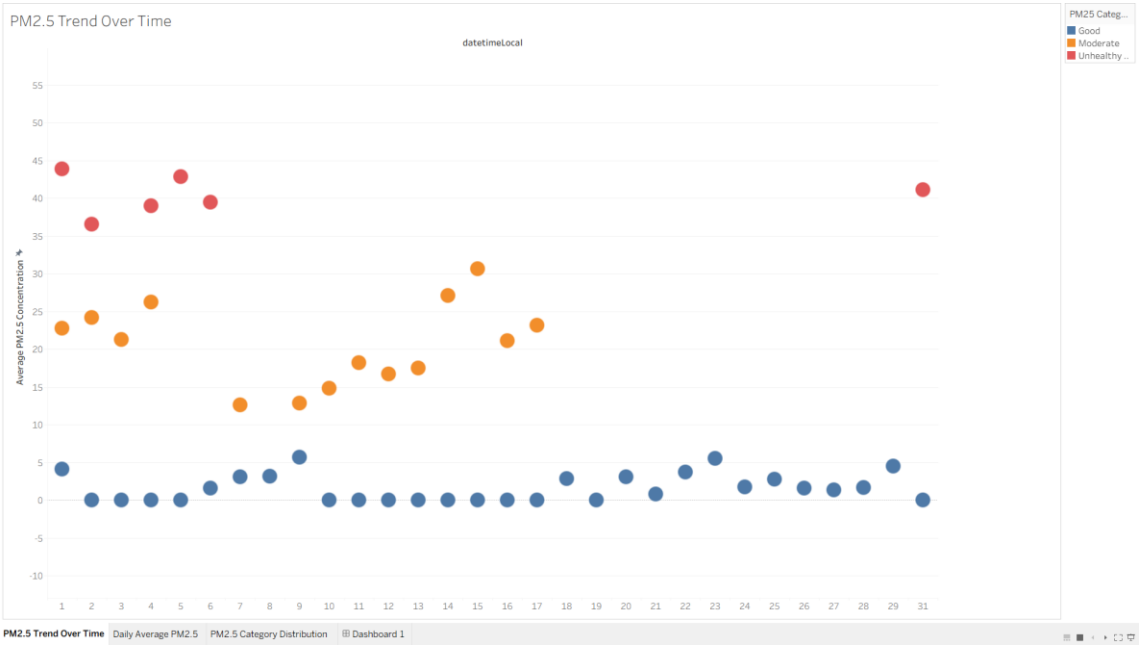
Google Colab for Analytics using Spark:
https://colab.research.google.com/drive/13hZj9HrfV9rYGsM7x5wkF9u9_vny0zZ1?usp=sharing



This dashboard visualizes PM2.5 air pollution data collected from monitoring stations in Brooklyn. The dashboard was made using Tableau to analyze pollution trends, daily air quality conditions, and the distribution of PM2.5 categories over time.

The visualizations help users understand fluctuations in air quality levels and identify periods with higher pollution concentrations.

1. PM2.5 Changes Over Time by Category



Description

- This is a line chart of the average concentration of PM2.5 particles over time, using daily measurements.

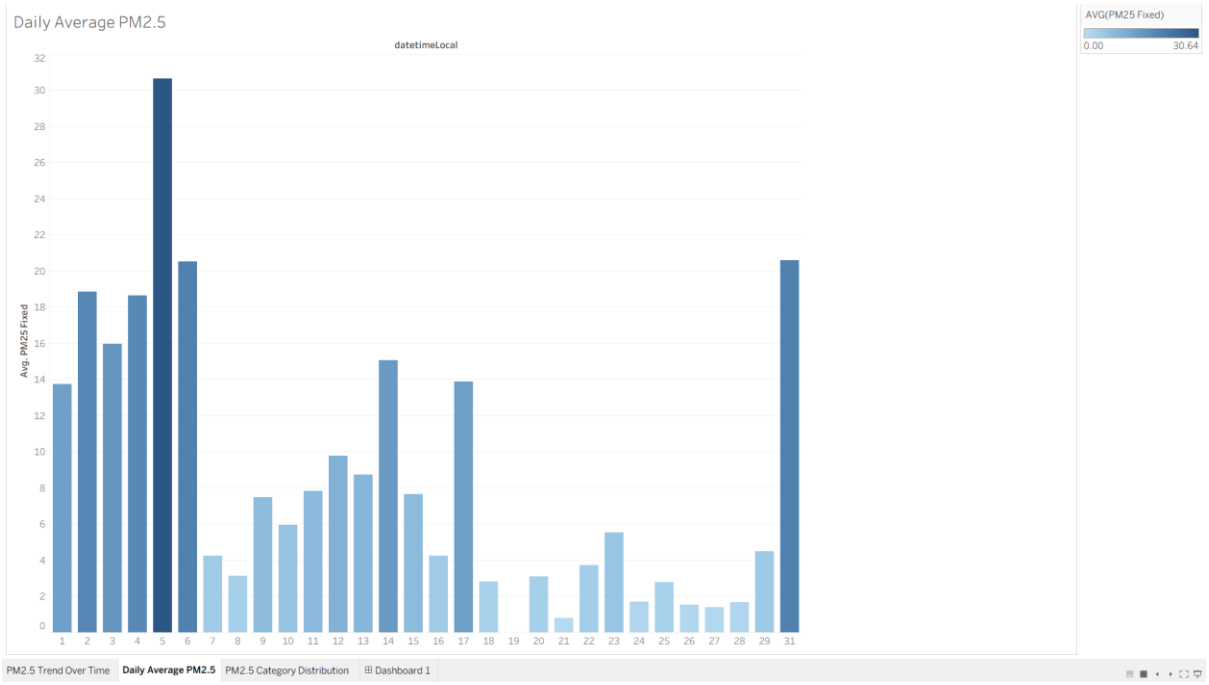
Insight

- The levels of PM2.5 particles vary from minute to minute during the observation period.
- The there are several peaks of pollution for certain days, which suggests temporary high level of air pollution.
- The graph also highlights periods of sharp reductions in PM2.5 concentrations, indicative of periods of improved air quality.

Interpretation

- The pattern of fluctuations suggests that the air quality is affected by the change in environmental and man-made factors. In order to determine periods of pollution peak and to facilitate environmental analysis, the trends of these values should be monitored.

2. Daily Average PM2.5



Description

- This bar chart shows daily average PM2.5 concentration in color intensity to indicate the level of pollution.

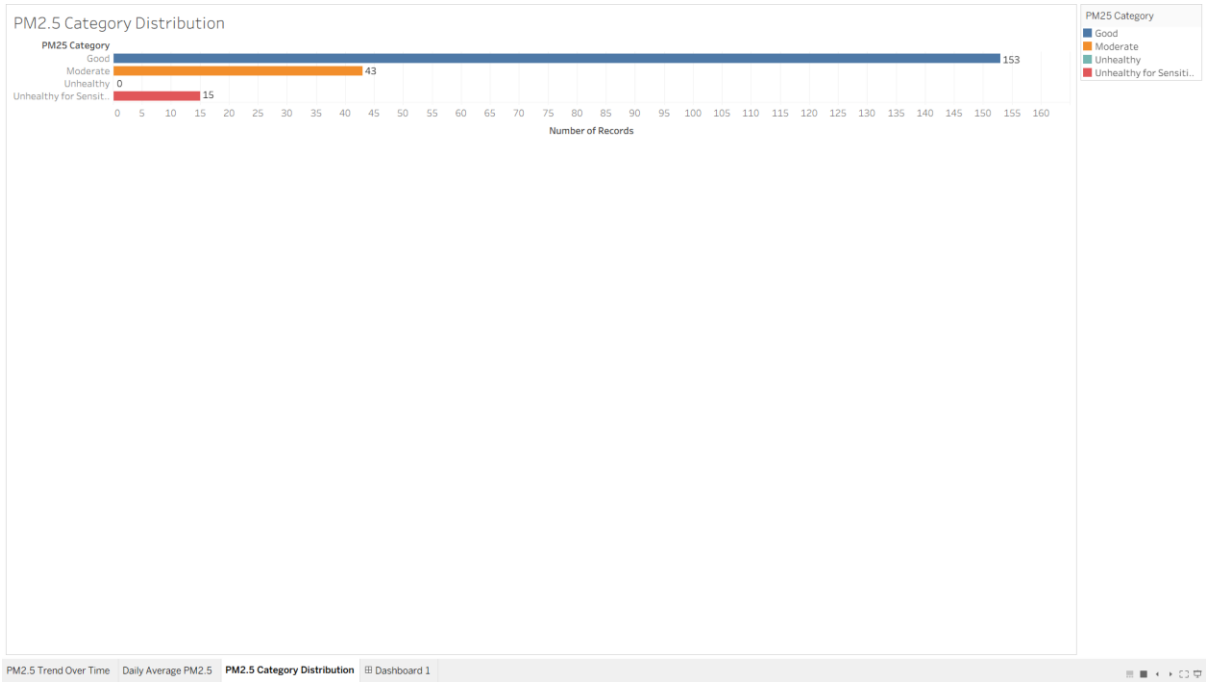
Insight

- The levels of PM2.5 were significantly higher on some days than on others.
- Moderation was the most common level of pollution during the monitoring period.
- A few days were rated at "unhealthy" pollution levels, which means that air quality may be harmful to sensitive groups.

Interpretation

- Users can use daily comparisons to see how specific times relate to increased exposure to pollution. This information can help inform decision making for environmental monitoring and public health awareness.

3. PM2.5 Category Distribution



Description

- This visualization displays the distribution of PM2.5 measurements by air quality categories (Good, Moderate, Unhealthy for Sensitive Groups, etc.)

Insight

- The majority of PM2.5 measurements are in the “Good” category.
- Some moderate pollution is still present in the data set.
- A smaller percentage of observations were unhealthy pollution.

Interpretation

- While most air quality readings are considered safe, but there are still moderate and unhealthy air quality readings suggests that air quality still needs to be monitored and prevented.

Overall Findings

- The air quality situation in the monitored area is highly variable in time.
- Although most of the measurements are classified as “Good”, there are still pollution spikes.

- Tableau visualization can be used to turn raw environmental data into interactive and understandable information.
- Visual analytics can help to speed up environmental monitoring and aid in data-driven decision making.

Conclusion

- The concentration of PM2.5 varied during the observation period, suggesting temporal variability in air quality.
- The daily average measurements indicate that there were several days with moderate to unhealthy air pollution levels, which may indicate temporary increases in air pollution exposure.
- The distribution of PM2.5 category indicates that the majority of observations were in the “Good” category, but moderate and unhealthy pollution periods were also observed during certain periods.
- Pollution trends, daily comparisons, and distributions of air quality are clearly presented and interactively displayed in Tableau, making it effective for environmental monitoring data visualization.
- Data visualization can be used to enhance understanding of environmental conditions and facilitate better monitoring and decision making.

D. Result and Discussion

1. Result & Business insights

The designed big data architecture provide an end to end infrastructure capable of handling continuous PM 2.5 sensors data from Brooklyn, New York, across all stages process from real time ingestion to interactive visualization. The combination HDFS and HBase ensure that both historical and real time sensor records are stored and accessible efficiently, while Apache Spark enable fast analytical processing large volume of time series data.

From a business perspective, this architecture actionable environmental insight that directly support public health decision making. Stakeholders like city environmental agencies can use the Tableau to identify high pollution zone, track seasonal air quality trend, and respond quickly to sudden PM 2.5 spikes, also help enabling more informed and timely interventions for resident of Brooklyn.

2. Challenges & limitations

The challenges in this architecture is about the complexity of integrating multiple big data component like HDFS, HBase, Spark, and Tableau into a stable pipeline, particularly in ensuring low latency data flow between ingestion and storage layer. This project based on kaggle dataset rather than live sensor, the real tim streaming behaviour of ingestion layer is simulated rather than truly operational, which limit the ability to validate the system.

A limitation is this architecture does not incorporate pedictive analytics or machine learning model, the system can only describe and monitor current and historical air quality rather than forecast future pollution.

3. Future Improvements

Some improvements can extend the capabilities of this architecture. The most impactful addition is integrating a machine learning pipeline built on top of Apache Spark to enable predictive modelling of PM 2.5 levels based on historical patterns, weather conditions, and seasonal trends.

Connecting the architecture to live OpenAQ API feeds instead of kaggle dataset can make the ingestion layer full of operational in real time. Expanding the sensor coverage beyond Brooklyn to other New York City would increase the system value for city wide air quality and smart city planning.

E. Conclusion

This project presents a simulation of big data architecture designed to process, store, analyze, and visualize IoT based air quality sensor data from Brooklyn, New York. Integration real time streaming ingestiong, distributed storage to HDFS and HBase, analytics with Apache Spark, and interactive visualization with Tableau. All the process deployed on public cloud environment. The architecture addresses the full lifecycle of environmental sensor data in a scalable, fault tolerant, and efficient manner.

Technology decision in this architecture was made to match the characteristics of continuous PM 2.5 time series data, from the streaming ingestion over batch processing to the

combination of HDFS and HBase for storage. Storage and Monitoring layer ensures the system remains reliable, protected, and observable in operation.

The current implementation is based on open source dataset and serves as an architectureal design rather than fully deployed production system, it demonstrates a solid and extensible foundation for real world smart city monitoring. With future enhancements like live data integration and predictive machine learning capabilities, this architecture has the potential to serve as a meaningful infrastructure for data driven public health and urban environmental management.

F. Declaration Statement

AI Assistance Declaration

- Assisting the report for the use of better grammar
- Help brainstorming to design the architecture
- Help with the Colab Notebook Code (Not All)

Reference

Arshdeep Bahga & Vijay Madisetti (2019). Big Data Analytics: A Hands-On Approach. India: VPT.

[1] Acceldata, “HDFS: Key to Scalable and Reliable Big Data Storage,” 2025. [Online]. Available: https://www.acceldata.io/blog/why-hdfs-remains-vital-for-scalable-reliable-data-storage-in-2025.

[2] Acceldata, “Hadoop Architecture: Scalable Big Data Processing Framework,” 2026. [Online]. Available: https://www.acceldata.io/blog/hadoop-architecture-a-comprehensive-guide.

[3] Amazon Web Services, “What is Spark? – Introduction to Apache Spark and Analytics,” AWS, 2025. [Online]. Available: https://aws.amazon.com/what-is/apache-spark/.

[4] S. Anand et al., “End-to-End Architecture for Real-Time IoT Analytics and Predictive Maintenance,” *PMC*, 2025. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12074242/.

[5] CelerData, “Apache HBase,” 2024. [Online]. Available: https://celerdata.com/glossary/apache-hbase.

[6] CIO, “How to Secure Big Data in Hadoop,” 2023. [Online]. Available: https://www.cio.com/article/286526/big-data-how-to-secure-big-data-in-hadoop.html.

[7] S. Eswaran et al., “High Performance and Fault Tolerant Distributed File System for Big Data Storage and Processing Using Hadoop,” *ResearchGate*, 2014. [Online]. Available: https://www.researchgate.net/publication/286812890.

[8] D. Namiot et al., “Big Data Analytics on Apache Spark,” *International Journal of Data Science and Analytics*, Springer, 2016. [Online]. Available: https://link.springer.com/article/10.1007/s41060-016-0027-9.

[9] OpenLogic, “Hadoop Monitoring: Tools, Metrics, and Observability,” 2024. [Online]. Available: https://www.openlogic.com/blog/hadoop-monitoring-tools-observability.

[10] OpenLogic, “How to Improve Hadoop Security: Essential Best Practices,” 2025. [Online]. Available: https://www.openlogic.com/blog/hadoop-security-best-practices.

[11] M. Stanic et al., “Hadoop Oriented Smart Cities Architecture,” *MDPI Sensors*, vol. 18, no. 4, 2018. [Online]. Available: https://www.mdpi.com/1424-8220/18/4/1181.

[12] N. Zulkarnain et al., “Evaluation of Distributed Stream Processing Frameworks for IoT Applications in Smart Cities,” *Journal of Big Data*, Springer, vol. 6, no. 1, 2019. [Online]. Available: https://link.springer.com/article/10.1186/s40537-019-0215-2.

Turnitin Result for AI Detection: